

Pytorch 입문 부트캠프

2. Pytorch Fundamentals (2) Neural Network 학습

목차

- 1 Loss Function : MSE, Cross Entropy
- 2 Loss 최적화 : Gradient Descent, Backpropagation

1

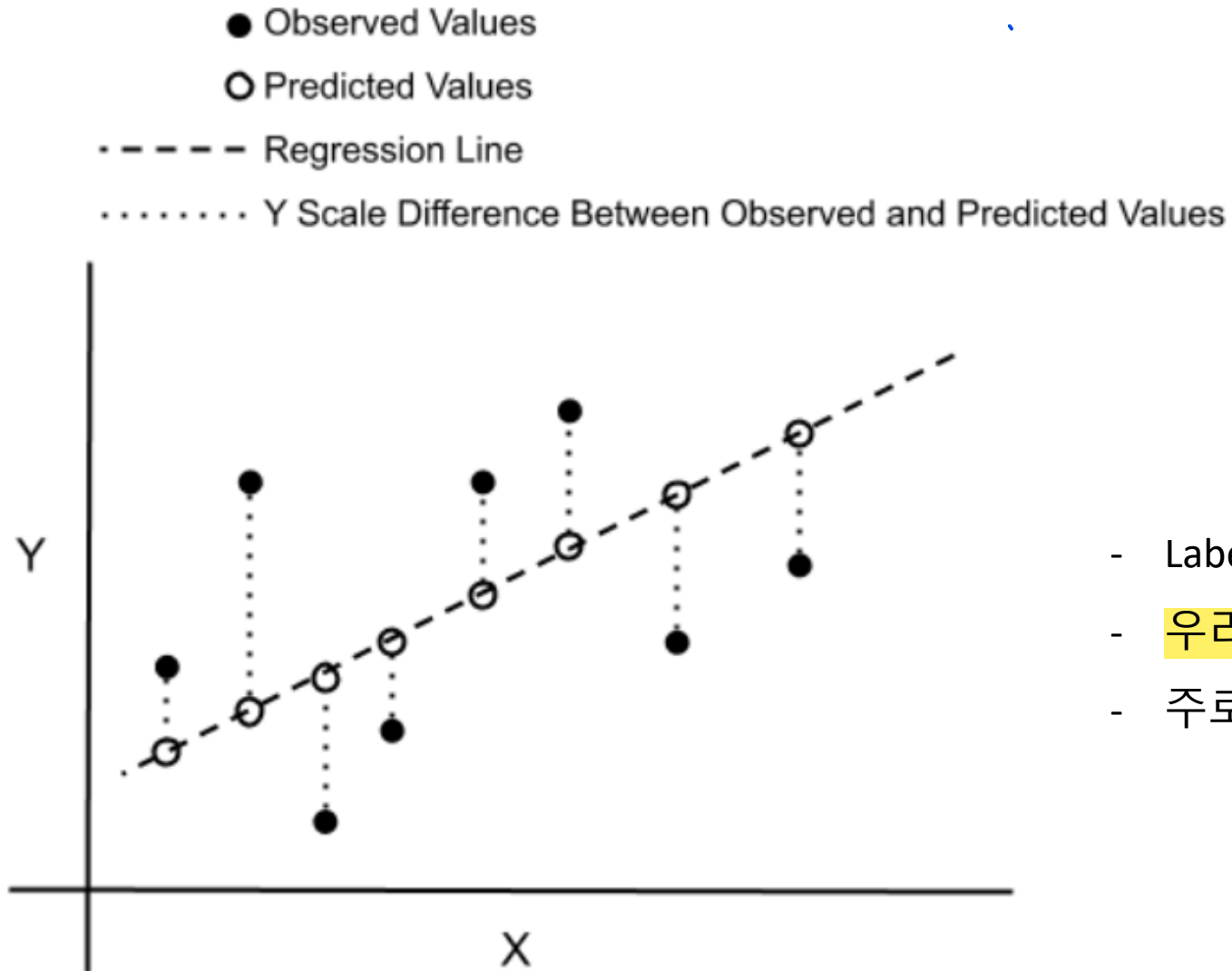
Loss Function : MSE, Cross Entorpy

Loss Function : MSE, Cross Entropy

- Loss 란?
 - Error , Loss는 모델의 예측값과 실제값 간의 차이
→ 이 차이를 계산하는 함수를 loss function
 - 모델 학습의 목적은 이 Loss function을 최소화하는 가중치(parameter)를 찾는 것
→ Loss Function을 Objective Function 이라고도 함
cost fun
 - 대표적으로 사용되는 Loss function : MSE, Cross Entropy

Loss Function : MSE, Cross Entropy

- MSE(Mean Squared Error) *회귀*



$$MSE = \frac{1}{n} \sum_{i=1}^n (\hat{Y}_i - Y_i)^2$$

Pred (pointing to $\hat{Y}_i$) *Real* (pointing to Y_i)

- Label의 실제값(Y) 와 예측값($\hat{Y}$)의 차이의 제곱합
- **우리의 목표는 이 MSE를 줄이는 것!**
- 주로 Regression 문제에 많이 사용되는 Loss

Loss Function : MSE, Cross Entropy

• Cross Entropy 분류

- 다음 2개 모형 중 어떤 것이 좋은 모형일까?
 - A 모형 : X 학생은 합격할 확률이 85%(불합격 확률은 15%)
 - B 모형 : X 학생은 합격할 확률이 51%(불합격 확률은 49%)
- 실제 결과에서 X학생이 합격하였다면 A가, 불합격 하였다면 B가 좋은 모형
- 정답을 맞출 확률이 높은 쪽이 좋은 모형 : Likelihood, 가능도, 우도

학생	합격	불합격
X1	1	0
X2	1	0
X3	0	1
X4	0	1

데이터의 합격 여부(실제값)



합격	불합격
0.1 ✓	0.9
0.7 ✓	0.3
0.4	0.6 ✓
0.8	0.2 ✓

A모형의 예측 확률

$$0.1 \times 0.7 \times 0.6 \times 0.2 = 0.0084$$

vs

합격	불합격
0.8 ✓	0.2
0.4 ✓	0.6
0.6	0.4 ✓
0.1	0.9 ✓

B모형의 예측 확률

$$0.8 \times 0.4 \times 0.4 \times 0.9 = 0.1152$$

<

Loss Function : MSE, Cross Entropy

- Cross Entropy

- 어떻게 하면 확률이 높은 모델을 만들 것인가?
- 각 데이터 샘플들의 (실제 값 x 예측 확률)을 모두 곱하여 계산
- 확률은 [0,1] 사이의 값이므로 계속 곱하면 0에 가까워짐 (데이터가 100개라면?)
 → log 함수를 통해 곱을 합으로 변환 ($\log(a \times b) = \log(a) + \log(b)$) / -로 양수로 전환

Log 함수 적용

- 로 양수로 전환

$0.1 \times 0.7 \times 0.6 \times 0.2 = 0.0084$	vs	$0.8 \times 0.4 \times 0.4 \times 0.9 = 0.1152$
$\ln(0.1) + \ln(0.7) + \ln(0.6) + \ln(0.2)$	vs	$\ln(0.8) + \ln(0.4) + \ln(0.4) + \ln(0.9)$
$\Rightarrow -2.3 \quad -0.36 \quad -0.51 \quad -1.61$		$\Rightarrow -0.22 \quad -0.91 \quad -0.91 \quad -0.1$
$-\ln(0.1) - \ln(0.7) - \ln(0.6) - \ln(0.2)$	vs	$-\ln(0.8) - \ln(0.4) - \ln(0.4) - \ln(0.9)$
$= 4.78$		$= 1.01$

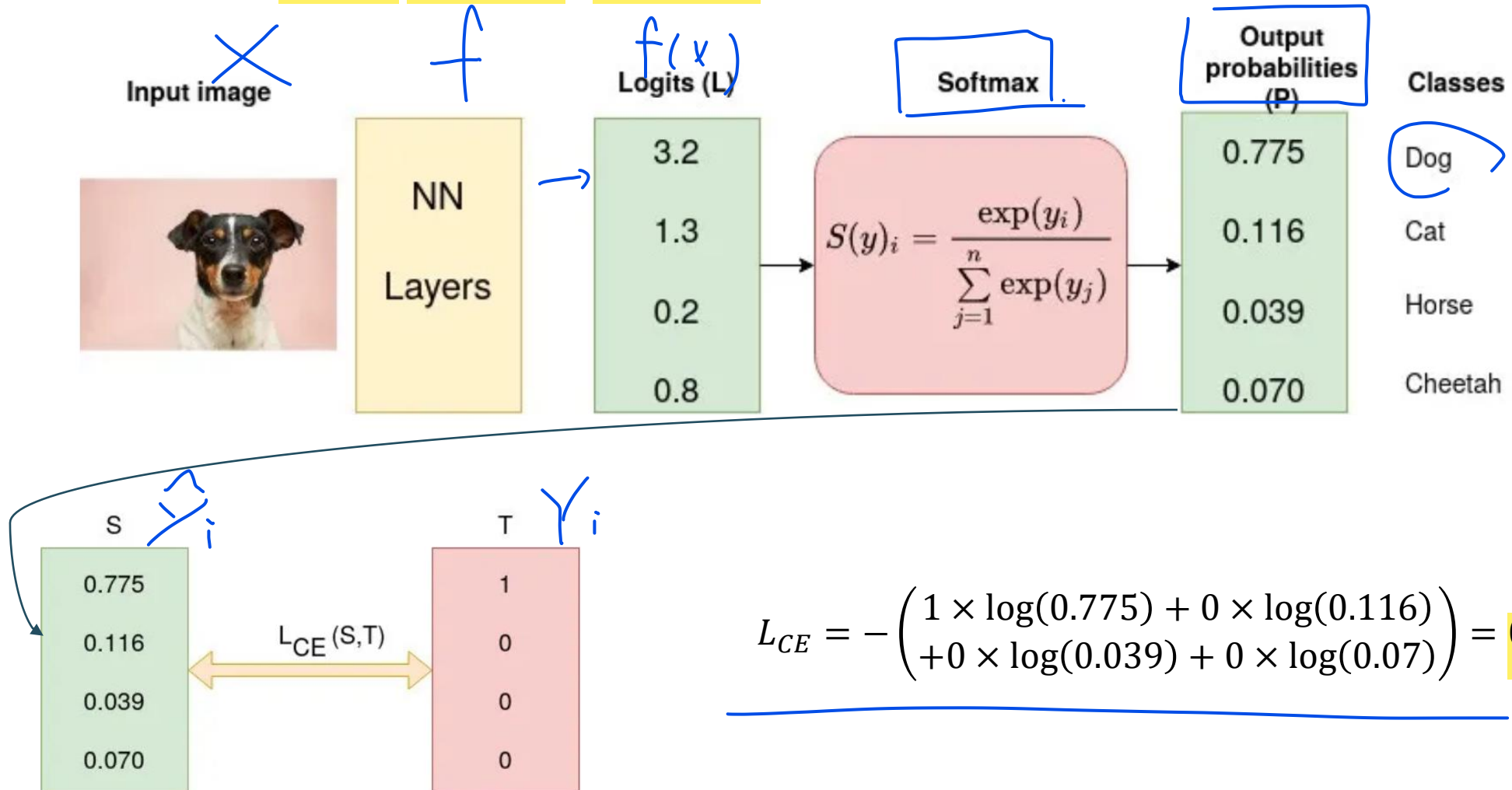
(Handwritten blue marks: checkmarks and underlines are present under the final results 4.78 and 1.01.)

$$\text{Cross Entropy} = - \sum_{i=1}^n y_i \ln(p(f(x_i))) + (1 - y_i) \ln(1 - p(f(x_i))), \quad \text{where } y_i = 1 \text{ or } y_i = 0$$

Loss Function : MSE, Cross Entropy

- Cross Entropy

- 실제 Cross Entropy 적용 시 계산 과정 : softmax 후의 확률 값으로 CE Loss 를 계산

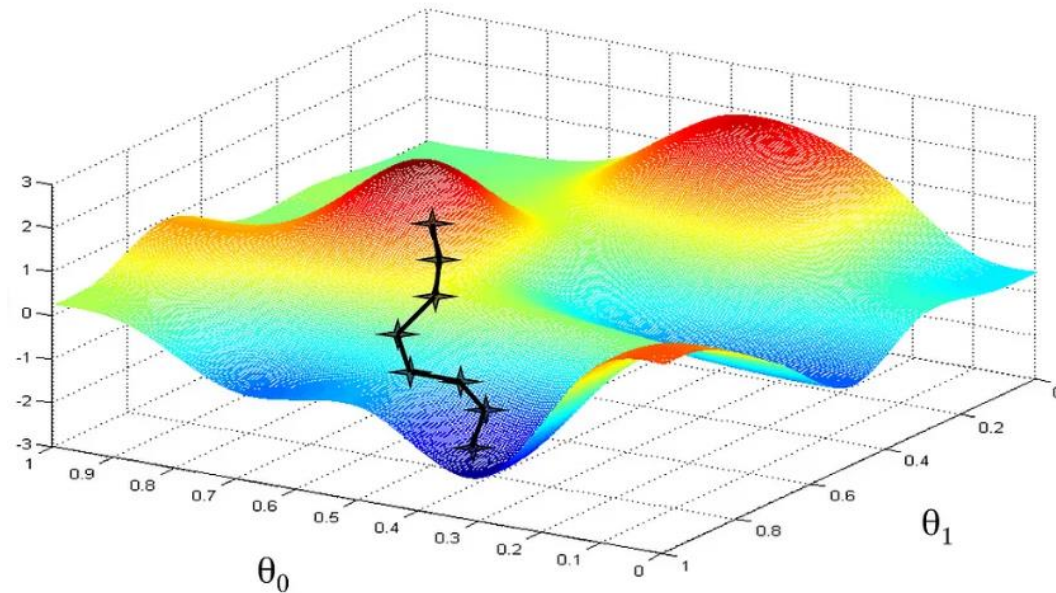


2

Loss 최적화: Gradient Descent Backpropagation

Loss 최적화: Gradient Descent

- Loss를 줄이는 것이 목표라면, 어떻게 최적화 할 수 있을까?
 - 산 정상에서 내려올 때 길을 모른다고 가정해보자
 - 빨리 내려오는 방법은 주변의 모든 방향으로 발을 뻗어봐서 가장 기울기가 가파른 방향으로 한걸음 내려가는 것
 - 한걸음 내려온 지점에서 또 기울기가 가장 가파른 방향으로 내려오는 것 (더 이상 주변에 내려갈 곳이 없을 때까지 반복= minimum에 도달할 때까지)



Loss 최적화: Gradient Descent

- Gradient Descent

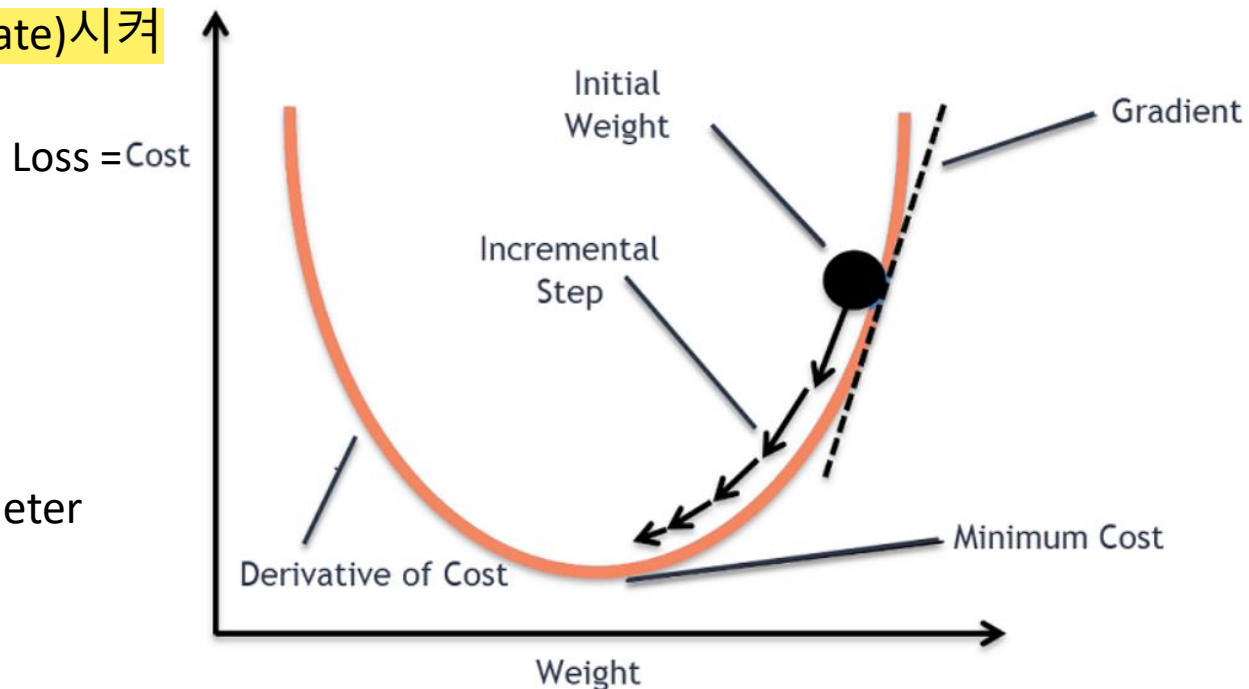
- Gradient : 기울기, Descent : 하강
- 주어진 Loss Function 의 최소값(minimum)을 찾아가는 알고리즘
- Gradient 의 정의
 - 어떤 함수가 가장 빠르게 증가하는 방향과 그 기울기(크기)를 나타내는 vector space
 - $y = f(x)$ 에서 x 의 값이 아주 조금 변할 때 $f(x)$ 의 값이 변하는 비율 (미분을 통해 계산)
- 결국, weight(=model parameter)를 조금씩 변화(update)시켜

Loss Function의 최소값으로 도달하는 것이 목표

$$W_{new} = W_{old} - \alpha \times (Gradient) = W_{old} - \alpha \times \frac{\partial L}{\partial W}$$

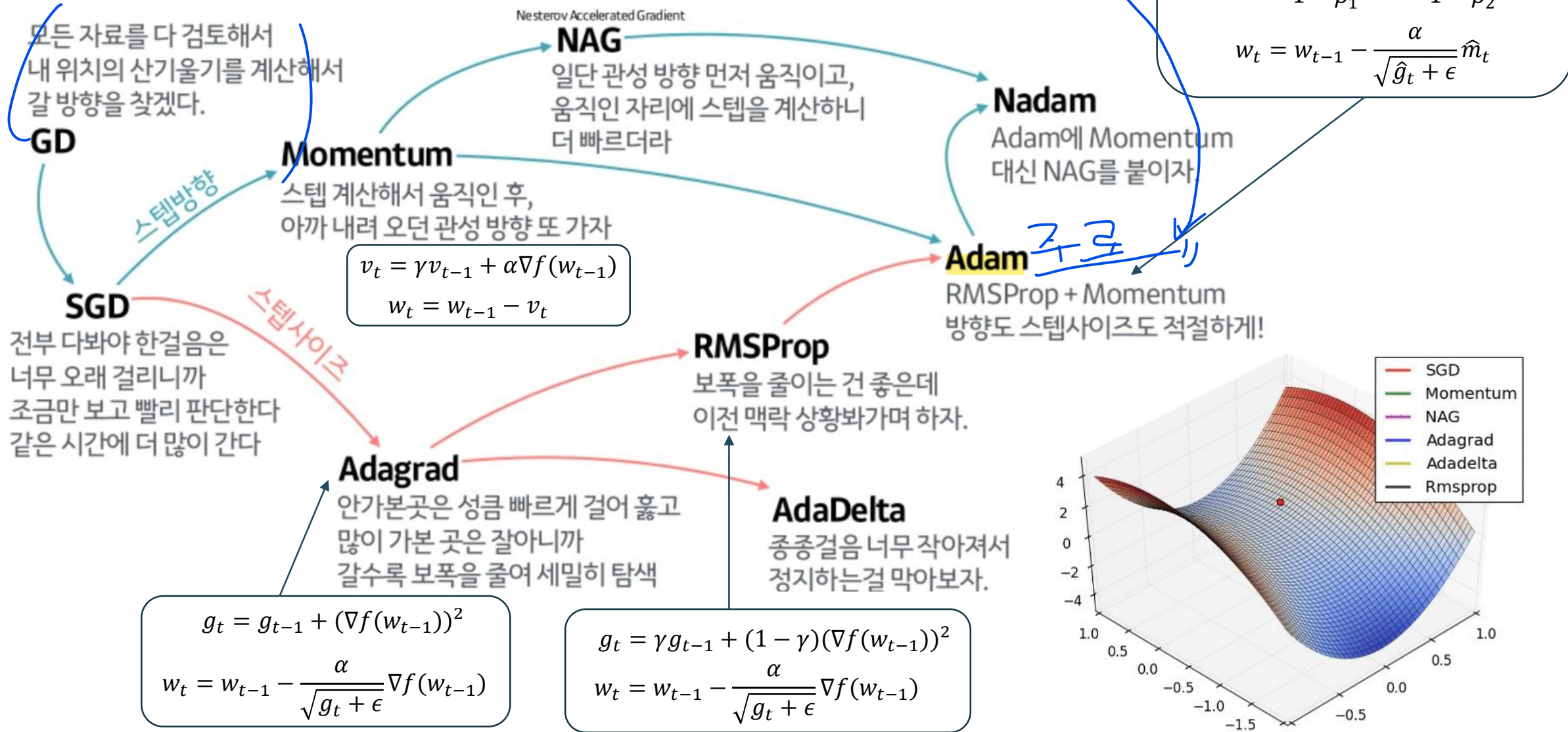
* α : Learning rate

Weight update 속도를 조절하는 hyper parameter



Loss 최적화: Gradient Descent

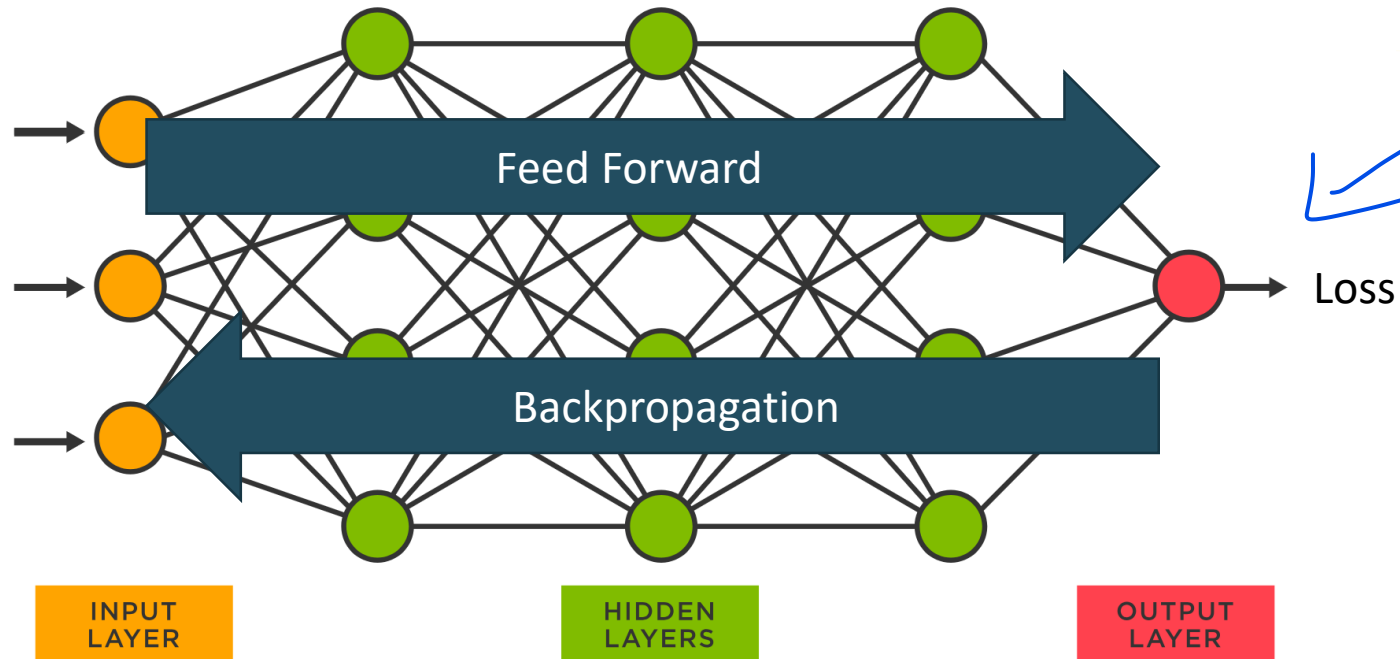
- Gradient Descent를 효율적으로 하기 위한 **방법들** (Optimization Techniques)



Loss 최적화: Backpropagation

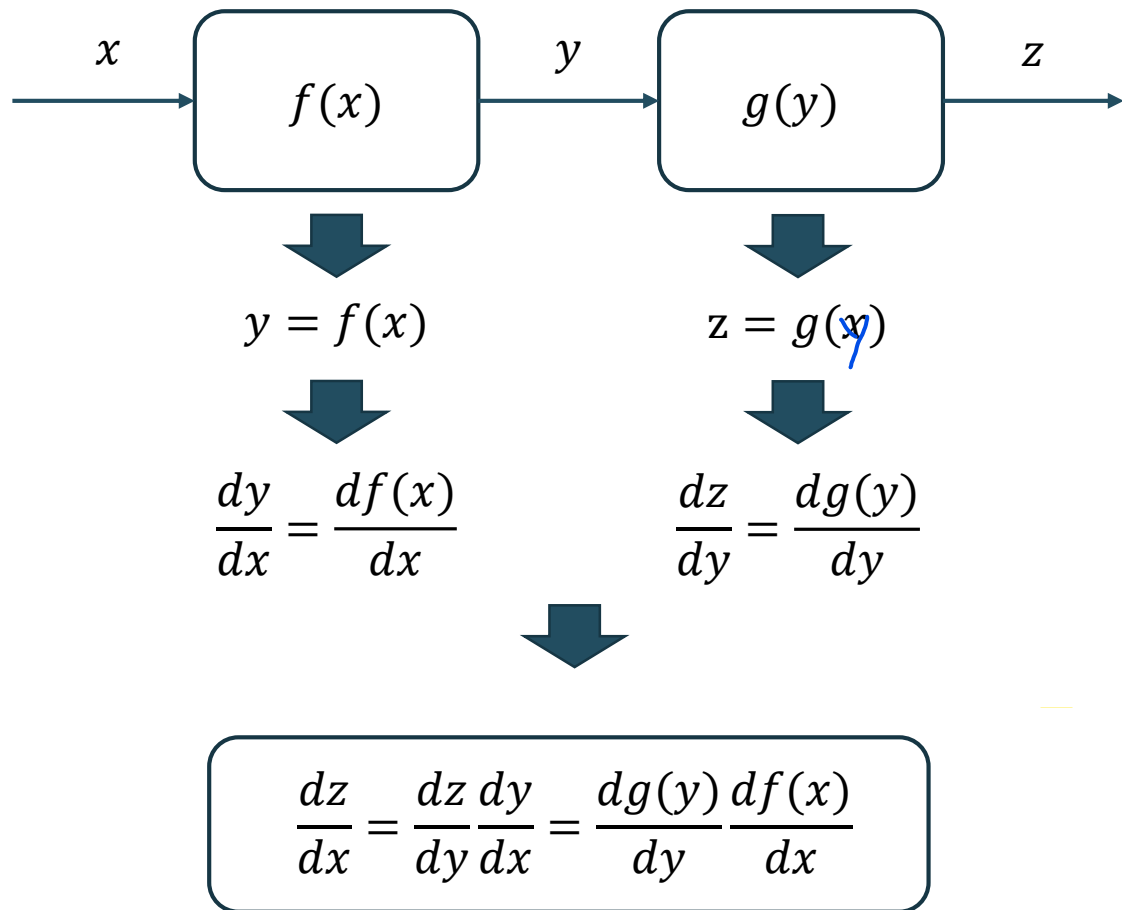
- Backpropagation

- 앞서 Gradient는 $\frac{\partial L}{\partial W}$ 라고 표현하였는데, 이 Gradient를 계산하기 위한 방법
- Input > hidden units(layers) > output 으로 구성된 Neural Net에서 정보가 전달되는 방향을 Feedforward
- Feedforward를 통해 output 예측값을 얻고($f(x)$) 이걸 실제 y 와 MSE, CE 등의 방식으로 비교하여 Loss를 계산하게 되면, Loss를 Weight로 편미분하여(=Gradient를 구하여) 그 반대 방향으로 weight를 update

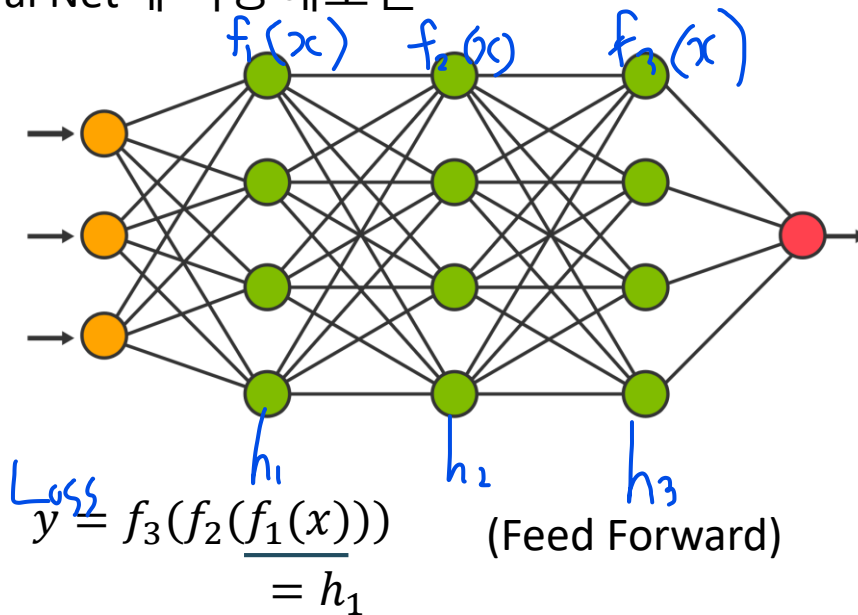


Loss 최적화: Backpropagation

- Chain Rule



- Neural Net에 적용해보면



$$\frac{dL}{dx} = \frac{dy}{dx} = \frac{df_3(h_2)}{dh_2} \frac{df_2(h_1)}{dh_1} \frac{df_1(x)}{dx}$$

$$= \prod_{n=1}^3 \frac{df_n(h_{n-1})}{dh_{n-1}}$$

Loss를 각 layer의 파라미터로 편미분, 그 과정을 각각 행렬연산으로 계산 (backpropagation)

실습 해볼까요?